



QUANTUM WALLET

The Complete Project Guide

A from-scratch walkthrough of the repository: what it is, why it exists, the system architecture, and every on-chain & off-chain component.

Post-Quantum Security · ERC-4337 Account Abstraction · ML-KEM / ML-DSA / FALCON · The Graph · IPFS

Repository: [SuhasRam356/Quantum-wallet](#)

Generated as a code-analysis study document

1. What is this project? (60-second answer)	4
2. The problem it solves: the quantum threat	4
Today's wallets	4
The threat	4
The NIST answer	5
3. Prerequisite concepts, explained simply	5
3.1 Wallets: EOA vs Smart Contract	5
3.2 Public-key cryptography	5
3.3 Why quantum computers break ECDSA	5
3.4 Post-Quantum Cryptography (PQC)	6
3.5 Account Abstraction & ERC-4337	6
3.6 CREATE2 & counterfactual addresses	6
3.7 The Graph & IPFS	7
4. QuantumSmartWallet	8
4.1 QuantumSmartWallet.sol	8
4.2 QuantumSmartWalletFactory.sol	10
4.3 MockFalconPrecompile.sol	11
5. QuantumSmartWalletFactory	12
5.1 src/utlis/pqcKeyManager.js — the crypto engine	11
5.2 src/hooks/useSmartWallet.js — the hook	12
5.3 API routes — the "relayer" / mini-bundler	12
5.4 The UI pages	13
6. QuantumSmartWalletFactory	14
6.1 Connecting & computing your wallet address	13
6.2 Sending ETH (the signature moment)	14
6.3 Generating & registering a PQC key	14
6.4 Uploading an encrypted file (the Vault)	14
6.5 Social recovery	15

- Real 16
- Mocked / simulated 16
- Design reality vs. naming 17
- 12. Bugs, drift & things to verify 17**
- 13. How to run it yourself 18**
 - Prerequisites 18
 - Frontend 18
 - Contracts (Sepolia) 19
 - Subgraph 19
 - Tests 19
- 14. Glossary 19**

1. What is this project? (60-second answer)

Quantum Wallet is a proof-of-concept (PoC) decentralized application (dApp) that demonstrates what a **post-quantum secure crypto wallet** could look like.

Instead of holding your funds in a normal externally-owned account (like a plain MetaMask address), it holds them inside a **smart contract wallet** (a "Smart Vault").

To authorize any transaction you must present **two** signatures:

- A normal **ECDSA** signature from your everyday wallet (like MetaMask), and
- A **post-quantum** signature (FALCON-512) whose public key is committed on-chain.

The project uses **ERC-4337 Account Abstraction** so that your normal, empty wallet can pay gas while the smart contract enforces the quantum-safe signature. It also includes:

- a **crypto-agility layer** (switch between PQC algorithms without redeploying),
- a **hybrid ML-KEM + AES-GCM encrypted file vault**,
- **social recovery** via guardians,
- a **The Graph subgraph** for fast indexing, and
- a polished **glassmorphic React dashboard**.

Important framing: the architecture is real and the lattice cryptography for the vault is real (ML-KEM-768, ML-DSA-65 via [@noble/post-quantum](#)), but the **on-chain FALCON-512 verification is mocked** with a simulated "precompile" contract. See Section 11.

2. The problem it solves: the quantum threat

Today's wallets

Ethereum (and Bitcoin) wallets rely on **Elliptic Curve Cryptography (ECC)** — specifically the `secp256k1` curve for Ethereum. Your **public key** is derived from your **private key** using elliptic-curve multiplication. It is easy to go private → public, but believed to be extremely hard to go public → private on a normal computer. This is the "Elliptic Curve Discrete Logarithm Problem."

The threat

In 1994, mathematician **Peter Shor** published **Shor's algorithm**, which proves that a sufficiently powerful **quantum computer** could solve the discrete-logarithm problem (and integer factorization, which breaks RSA) in polynomial time — i.e. minutes instead of billions of years.

If a large-scale quantum computer is ever built, anyone who can see your public key could recover your private key and drain your funds. ****Signature algorithms must be upgraded before that day comes.****

The NIST answer

The US **NIST** ran a multi-year competition and in **August 2024** standardized the first post-quantum algorithms:

FIPS	Name	Family	Used for
FIPS 203	ML-KEM (a.k.a. Kyber)	Lattice-based	Key encapsulation / encryption
FIPS 204	ML-DSA (a.k.a. Dilithium)	Lattice-based	Digital signatures
FIPS 205	SLH-DSA (a.k.a. SPHINCS+)	Hash-based	Digital signatures
FIPS 206	FN-DSA (a.k.a. FALCON)	Lattice-based	Digital signatures

This project references **FALCON-512** (signatures), **ML-DSA-65** (signatures) and **ML-KEM-768** (encryption) — three real lattice-based schemes.

3. Prerequisite concepts, explained simply

3.1 Wallets: EOA vs Smart Contract

- **EOA (Externally Owned Account):** your MetaMask account. Controlled directly by a private key. One signature = one transaction.
- **Smart Contract Wallet:** an account whose "owner" is actually a program running on the blockchain. You don't sign a raw transaction; you submit a request and the program's code decides whether it is valid. This lets you add features a plain EOA can't have: custom signature checks, multi-signature, social recovery, spending limits, etc.

Quantum Wallet is a smart-contract wallet. Your funds live at the contract's address, not your MetaMask address.

3.2 Public-key cryptography

- **Private key:** a secret only you know. Never shared.
- **Public key:** derived from the private key. Safe to share.
- **Signature:** proves you hold the private key for a given message, without revealing it.
- **Verification:** anyone with the public key can check the signature is valid.

A **hash** (like `keccak256`) is a one-way function that turns any input into a fixed-size "fingerprint." You can't reverse it, and changing one bit of input changes the output completely.

3.3 Why quantum computers break ECDSA

Normal computers must "guess" the discrete logarithm (exponential work). A quantum computer running Shor's algorithm can solve it efficiently. Symmetric crypto (AES) and hashing (SHA/keccak) are mostly safe — you just double key sizes (Grover's algorithm). The real breakage is in **asymmetric** crypto (RSA, ECDSA, Diffie-Hellman).

3.4 Post-Quantum Cryptography (PQC)

PQC = algorithms believed secure against both classical and quantum computers.

- **Lattice-based:** security rests on hard problems on high-dimensional lattices (e.g. "Learning With Errors", LWE). ML-KEM, ML-DSA, FALCON are all lattice-based.
- **Hash-based:** security rests on hash-function preimage resistance. SPHINCS+.
- **Code-based:** security rests on hard coding-theory problems (e.g. syndrome decoding).

The README discusses code-based schemes as an alternative (small signatures but huge keys).

Trade-offs that matter for this project:

Scheme	Public key size	Signature size
ECDSA (secp256k1)	64 bytes	~65 bytes
FALCON-512	897 bytes	666 bytes
ML-DSA-65	1952 bytes	~3309 bytes

PQC keys/signatures are huge compared to ECDSA. That is why storing and verifying them on-chain is expensive — a central design challenge this project wrestles with.

3.5 Account Abstraction & ERC-4337

ERC-4337 is the standard for account abstraction without changing the Ethereum protocol.

It introduces:

- **UserOperation (UserOp):** a pseudo-transaction object with fields like `sender`, `nonce`, `callData`, gas limits/fees, and `signature`. (In v0.7, fields `accountGasLimits` and `gasFees` are packed into `bytes32` values — hence `PackedUserOperation`.)
- **EntryPoint:** a singleton smart contract that validates and executes UserOps. On Sepolia its canonical address is `0x0000000071727de22e5e9d8baf0edac6f37da032`.
- **Bundler:** an off-chain service that collects UserOps and submits them to the EntryPoint (like a miner for UserOps). This project acts as its own minimal bundler.
- **Paymaster (optional):** a contract that sponsors gas. Not implemented yet (roadmap).

The flow: a UserOp is validated by the wallet's `validateUserOp`, then executed by the wallet's `execute`. The **wallet contract itself** implements the `IAccount` interface.

Key insight for this project: because PQC signatures are huge, the project does NOT put the PQC signature into a normal transaction's signature field. Instead the PQC data rides inside the UserOp's `signature` bytes, and the smart wallet verifies it during `validateUserOp`.

3.6 CREATE2 & counterfactual addresses

`CREATE2` lets you deploy a contract to an address that is **predictable before deployment**.

The address depends only on: the deployer (factory) address, a `salt`, and the hash of the creation bytecode.

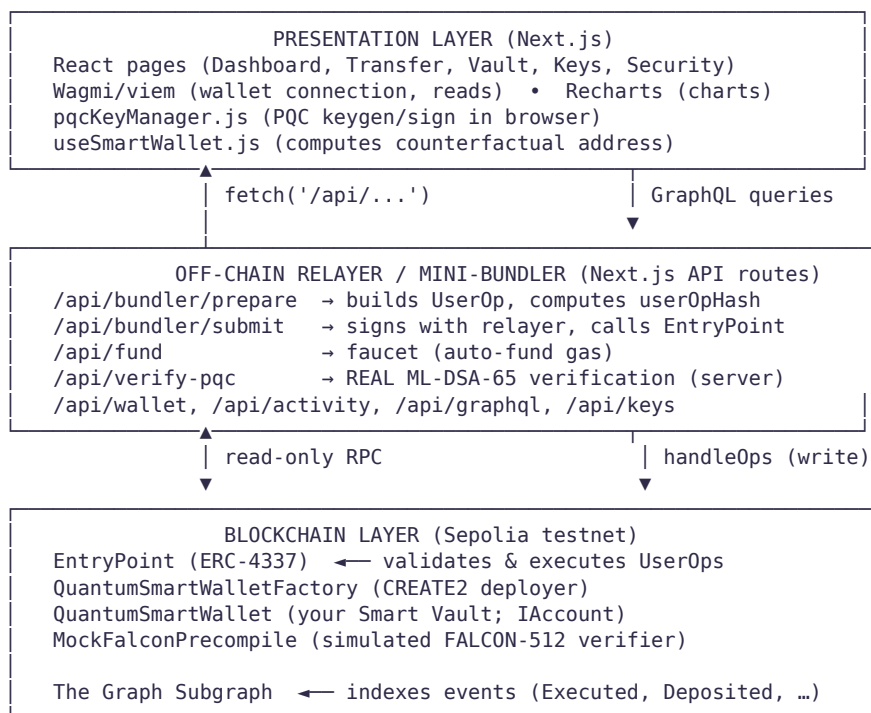
`getAddress()` computes this address without deploying anything. This is how the app shows you a "your smart wallet address" before the wallet even exists ("counterfactual address").

3.7 The Graph & IPFS

- **The Graph:** a decentralized indexing protocol. A **subgraph** watches smart-contract events and writes them into a queryable store (GraphQL). The dApp queries the subgraph instead of scanning the whole chain — much faster.
- **IPFS:** a content-addressed, decentralized file system. Files are identified by their content hash (**CID**), not a location. The project references IPFS CIDs for identity and vault files.

4. System architecture (the big picture)

The project is split into three layers, exactly as the README says:



Design decisions worth noticing:

- **Funds live in the contract, not the EOA.** Your MetaMask is only a "gas payer / second factor."
- **Dual-signature model.** ECDSA (2FA) + PQC (quantum-safe) are both required.
- **PQC key is stored on-chain only as a hash** (`pqcPubKeyHash = keccak256(pubkey)`), a compact "commitment." The full key is only sent when needed.
- **Crypto-agility** via `pqcAlgorithmId` — swap the algorithm by changing one `uint8`.
- **The heavy crypto (PQC keygen/sign) happens client-side**, so private keys never touch the server.

5. The repository layout

```

Quantum-wallet/
├── contracts/                # Solidity smart contracts
│   ├── QuantumSmartWallet.sol
│   ├── QuantumSmartWalletFactory.sol
│   └── MockFalconPrecompile.sol
├── src/                      # Next.js app (frontend + API routes)
│   ├── app/
│   │   ├── layout.js        # root layout (nav + provider)
│   │   ├── page.js          # Dashboard
│   │   ├── globals.css      # design tokens + glassmorphism
│   │   ├── keys/page.js     # PQC key generation/registration
│   │   ├── transfer/page.js # send/receive
│   │   ├── vault/page.js    # encrypted file vault
│   │   ├── security/page.js # guardians + logs
│   │   └── api/             # the off-chain relay routes
│   │       ├── bundler/prepare/route.js
│   │       ├── bundler/submit/route.js
│   │       ├── fund/route.js
│   │       ├── verify-pqc/route.js
│   │       ├── wallet/route.js
│   │       ├── activity/route.js
│   │       ├── graphql/route.js
│   │       └── keys/route.js
│   ├── components/          # NavBar, Web3Provider, charts
│   ├── hooks/useSmartWallet.js
│   └── utils/
│       ├── constants.js     # deployed addresses + ABIs
│       └── pqcKeyManager.js # PQC keygen/sign/encrypt helpers
├── subgraph/                # The Graph subgraph
│   ├── schema.graphql
│   ├── subgraph.yaml
│   ├── abis/
│   └── src/mapping.ts
├── scripts/                 # deploy & helper scripts
├── test/                   # Hardhat JS tests
├── testFoundry/            # Foundry Solidity tests
├── artifacts/ cache/       # Hardhat build output (generated)
├── hardhat.config.js       # Hardhat config
├── foundry.toml            # Foundry config
├── .github/workflows/      # CI (tests + Slither)
└── package.json

```

6. On-chain layer — deep dive

6.1 QuantumSmartWallet.sol

This is the heart of the project. It implements the ERC-4337 `IAccount` interface and acts as the "Smart Vault."

State variables

```

address public immutable entryPoint; // the ERC-4337 EntryPoint (only caller allowed)
uint8 public pqcAlgorithmId;         // 1 = ML-DSA, 2 = FALCON-512 (crypto-agility!)
bytes32 public pqcPubKeyHash;        // keccak256 commitment of the PQC public key
address public owner;                // the ECDSA owner (your MetaMask address)
address public pqcPrecompile;        // the (mock) FALCON-512 verifier address

```

The interesting part is `pqcAlgorithmId`: it makes the wallet **crypto-agile**. The verification logic branches on this ID, so you could upgrade the algorithm without

redeploying the vault.

It also stores social-recovery state:

```
string public userIdentity;           // IPFS CID of your profile
string[] public vaultFiles;          // IPFS CIDs of your encrypted files
address[] public guardiansList;      // trusted guardians
mapping(address => bool) public isGuardian;
struct RecoveryProposal { ... };      // pending recovery (new owner/key)
```

Constructor

```
constructor(address _entryPoint, uint8 _pqcAlgorithmId, bytes32 _pqcPubKeyHash,
             address _initialOwner, address _pqcPrecompile) {
    entryPoint = _entryPoint;
    pqcAlgorithmId = _pqcAlgorithmId;
    pqcPubKeyHash = _pqcPubKeyHash;
    owner = _initialOwner;
    pqcPrecompile = _pqcPrecompile;
}
```

Sets the 5 immutable/mutable fields. Note the PQC key is stored as a **hash**, not the full 897-byte key — saving massive storage gas.

receive() — receiving ETH

```
receive() external payable { emit Deposited(msg.sender, msg.value); }
```

Anyone can send ETH directly to the vault; it emits a `Deposited` event (which the subgraph indexes).

validateUserOp(...) — the security gate

This is called by the EntryPoint before execution. It returns:

- `0` → success,
- `1` → signature failure,
- or a packed "aggregator + timelock" value in more advanced cases.

Step by step, the code:

- **Length check:** `userOp.signature` must be $\geq$ **1628 bytes**

= 65 (ECDSA) + 897 (FALCON pk) + 666 (FALCON sig).

- **ECDSA recovery:** `hash = toEthSignedMessageHash(userOpHash)` then

`hash.recover(userOp.signature[0:65])` must equal `owner`. This is the classic

"2FA" check — proving the MetaMask owner approved the UserOp.

- **PQC branch (crypto-agility):**

- If `pqcAlgorithmId == 2` (**FALCON-512**): extract `falconPubKey` (bytes 65–962) and

`falconSig` (bytes 962–1628). Verify `keccak256(falconPubKey) == pqcPubKeyHash`

(the on-chain commitment). Then `staticcall` the precompile with

`abi.encodePacked(hash, falconPubKey, falconSig)`. If it returns `true`, pass.

- If `pqcAlgorithmId == 1` (**ML-DSA**): `revert("ML-DSA precompile not yet available...")`

— explicitly not wired up yet.

- Else: fail (unsupported algorithm).

- **Pay for itself:** if `missingAccountFunds > 0`, the vault pays the EntryPoint so it can

cover its own gas (deposit-pay mechanism).

The `requireEntryPoint` modifier guarantees only the `EntryPoint` can call these sensitive functions — preventing anyone from calling `execute` directly.

`execute(...)` — the action

```
function execute(address target, uint256 value, bytes calldata data)
    external requireEntryPoint returns (bytes memory) {
    (bool success, bytes memory result) = target.call{value: value}(data);
    require(success, "Transaction execution failed");
    emit Executed(target, value, data);
    return result;
}
```

Only the `EntryPoint` can call it (and the `EntryPoint` only calls it after `validateUserOp` passed). It forwards a call to any `target` with any `value/data` — so the wallet can send ETH or interact with any other contract.

PQC key management

```
function setPqcPublicKey(uint8 newAlgorithmId, bytes32 newHash) external requireEntryPoint {
    pqcAlgorithmId = newAlgorithmId;
    pqcPubKeyHash = newHash;
    emit PqcKeyUpdated(newAlgorithmId, newHash);
}
```

Rotates the PQC key/algorithm — also gated behind the `EntryPoint` (so it requires a valid `UserOp` with valid signatures first).

IPFS & identity functions

```
function setIdentity(string calldata cid) external requireEntryPoint { ... } // link profile
function addVaultFile(string calldata cid) external requireEntryPoint { ... } // add file ref
function addGuardian(address guardian) external requireEntryPoint { ... } // add guardian
```

Social recovery

```
initiateRecovery(newOwner, newAlgorithmId, newPubKeyHash) // guardian starts a proposal
approveRecovery() // other guardians approve
executeRecovery() // after 1-day timelock + majority
```

The recovery flow:

- A **guardian** calls `initiateRecovery` proposing a new owner + new PQC key.
- The proposal gets a **1-day timelock** (`executeAfter = now + 1 days`).
- Other guardians call `approveRecovery`.
- Once `approvalCount >= (guardiansList.length / 2) + 1` **and** the timelock passed,

anyone calls `executeRecovery` to rotate owner + PQC key.

This is a simplified PoC (the code even notes a production version would use a `recoveryNonce` to clean state between proposals).

6.2 QuantumSmartWalletFactory.sol

A **CREATE2 factory** that deploys `QuantumSmartWallet` instances.

- `createAccount(owner, pqcAlgorithmId, pqcPubKeyHash, salt):`
 - computes the counterfactual address via `getAddress`,
 - if code already exists there, just returns it,

- otherwise encodes the creation bytecode + constructor args and does

```
Create2.deploy(0, salt, bytecode),
```

- emits `AccountCreated(account, owner)`.

- `getAddress(owner, pqcAlgorithmId, pqcPubKeyHash, salt)`: computes the address using the CREATE2 formula (`0xff ++ factory ++ salt ++ keccak256(bytecode)`) **without deploying**.

The factory's constructor takes `(entryPoint, pqcPrecompile)` — both are baked into every wallet it deploys.

6.3 `MockFalconPrecompile.sol`

The "precompile" is **not** a real EVM precompile. It is an ordinary contract that **simulates** the behavior of EIP-7619-style FALCON verification for local testing.

Its `fallback()`:

```
if (msg.data.length != 1595) { _returnFalse(); return; } // 32 + 897 + 666
bytes32 messageHash = bytes32(msg.data[0:32]);
bytes32 embeddedHash = bytes32(msg.data[929:961]); // first 32 bytes of the sig
if (messageHash == embeddedHash) _returnTrue(); else _returnFalse();
```

In other words: it does **not** actually verify a FALCON signature. It just checks that the first 32 bytes of the "signature" equal the expected message hash — a stand-in so the whole pipeline can be exercised end-to-end without the (very heavy) real FALCON math on-chain.

7. Off-chain layer — deep dive

7.1 `src/utils/pqcKeyManager.js` — the crypto engine

This module does the client-side PQC work. Key points:

- `generateKeypair(password)` creates a mock FALCON-512 keypair with the correct

byte sizes: private key **1281 bytes**, public key **897 bytes**. It:

- tries to fetch quantum-random entropy from the ANU QRNG (falls back to WebCrypto),
- mixes it into random key bytes,
- also generates a **real ML-KEM-768 keypair** (`mL_kem768.keygen()`),
- encrypts the private keys with **PBKDF2 + AES-256-GCM** using a user password,
- stores the result in `localStorage` (public key + ML-KEM public key + encrypted blob).
- `signPayload(message)` prompts for the keystore password, decrypts the private keys,

then produces a **666-byte mock signature** and — critically for the mock precompile —

embeds `keccak256(message)` **in the signature's first 32 bytes**. This is not real FALCON signing; it exists so the mock precompile's "check" can pass.

- `getStoredKeypair()` / `getPublicKeyHex()` / `clearKeypair()` read/write/delete the

keypair from `localStorage`.

This is one of the most important files to understand the "real vs mock" split: key sizes match FALCON-512, ML-KEM is real, but the FALCON signature itself is a mock.

7.2 `src/hooks/useSmartWallet.js` — the hook

Computes your smart wallet's **counterfactual address**:

- Reads the connected EOA `address`.
- Uses a **placeholder** PQC key hash (`keccak256("placeholder_pqc_key_pending_registration")`)

so the address is stable before you generate a real key.

- Calls `factory.getAddress(owner, placeholderHash, salt=0)` via `publicClient.readContract`.
- Returns `smartWalletAddress` for the rest of the UI to use.

See Section 12: this call passes 3 arguments, but the current factory's `getAddress` takes 4 (owner, `uint8` algorithmId, bytes32 hash, salt) — the ABI in `constants.js` has drifted from the contract source.

7.3 API routes — the "relayer" / mini-bundler

These Next.js API routes are the off-chain glue that plays the role of an ERC-4337

bundler (and a faucet).

`/api/bundler/prepare`

Builds a `PackedUserOperation` and returns `{ userOp, userOpHash }`:

- Reads `sender, target, value, data, owner, pqcPubKeyHash, rawCallData`.
- Gets the wallet's nonce from `EntryPoint.getNonce(sender, 0)`.
- Checks `getCode(sender)`; if the wallet isn't deployed yet, builds `initCode` =

factory address + encoded `createAccount(...)` call (so the very first UserOp deploys the wallet).

- Builds `callData`: either the raw call (for `setIdentity/addVaultFile/etc.`) or wraps `execute(target, value, data)`.

- Fills in **hard-coded gas limits/fees** (a real bundler would call `eth_estimateUserOperationGas`).

- Computes `userOpHash` via `EntryPoint.getUserOpHash(opTuple)`.

`/api/bundler/submit`

Finalizes and submits the UserOp:

- Reads `{ userOp, pqcSignature, pqcPublicKey }`.
- Recomputes `userOpHash`.
- **Concatenates** the wallet's final signature:

```
userOp.signature = ECDSA(65) + FALCON_pk(897, padded) + FALCON_sig(666, padded).
```

- Connects the EntryPoint with the **relayer wallet** (from `SEPOLIA_PRIVATE_KEY`) and calls `handleOps([opTuple], relayerAddress)` — i.e. it is the bundler.

- Waits for the receipt and returns the `txHash`.

This is the "relayer system" the README mentions: your MetaMask stays empty of gas; the server's relayer key pays for the transaction on-chain.

`/api/fund`

A **faucet**: sends `0.005 ETH` from the relay to the requested address, with a simple in-memory 24-hour rate limit per address.

`/api/verify-pqc`

The **one genuinely real PQC-verification endpoint**: imports `ml_dsa65` from `@noble/post-quantum/ml-dsa.js` and calls `ml_dsa65.verify(sig, msg, pubKey)` server-side. Returns `{ valid, algorithm: 'ML-DSA-65 (FIPS-204)' }`.

`/api/wallet` **and** `/api/graphql`

Both try to read `Executed/Deposited` events from a **local node at** `127.0.0.1:8545` (and `/api/wallet` falls back to a hard-coded mock portfolio if the node is down). These are effectively legacy/demo data sources; the dashboard's real data comes from `/api/activity` (subgraph) when a wallet is connected.

`/api/activity`

Proxies the **The Graph subgraph** (hard-coded URL) and returns the wallet's transactions.

`/api/keys`

Returns static/mock key data — leftover demo endpoint, not used by the modern Keys page.

7.4 The UI pages

Page	Purpose
<code>page.js</code> (Dashboard)	Portfolio value,
<code>transfer/page.js</code>	Send/receive. R
<code>keys/page.js</code>	Generate PQC k
<code>vault/page.js</code>	Hybrid-encrypt
<code>security/page.js</code>	Add guardians,
<code>layout.js</code>	Root shell: <font
<code>components/Web3Provider.js</code>	Configures Wag
<code>components/NavBar.js</code>	Connect/disconn

Styling: `globals.css` defines the dark **glassmorphic** design tokens (glass cards, neon accents cyan/purple/green, gradient text). `page.module.css` is leftover Next.js boilerplate and is effectively unused.

8. End-to-end flows (step by step)

8.1 Connecting & computing your wallet address

- User clicks **Connect Wallet** → Wagmi's `injected` connector opens MetaMask (Sepolia).
- `useAccount()` returns the EOA `address`.

- `useSmartWallet()` calls `factory.getAddress(owner, placeholderHash, salt=0)` and displays the (counterfactual) smart wallet address.
- The dashboard queries `useBalance({ address: smartWalletAddress })` for the vault's balance and `/api/activity?address=...` for history.

8.2 Sending ETH (the signature moment)

- User fills recipient + amount on the Transfer page.
- `/api/bundler/prepare` builds the UserOp (deploys the wallet via `initCode` on the first use) and returns `{ userOp, userOpHash }`.
- **ECDSA step:** MetaMask signs `userOpHash` → 65-byte ECDSA signature.
- **PQC step:** `pqcKeyManager.signPayload(userOpHash)` decrypts the keystore and produces the 666-byte (mock) FALCON signature.
- `/api/bundler/submit` concatenates `ECDSA + FALCON_pk + FALCON_sig` into `userOp.signature` and submits via `EntryPoint.handleOps`.
- On-chain: `EntryPoint` calls `validateUserOp` →
 - recovers ECDSA → must equal `owner`,
 - checks `keccak256(pk) == pqcPubKeyHash`,
 - `staticcalls` the (mock) FALCON precompile → `true`,
 - returns `0` (valid).
- `EntryPoint` calls `execute(target, value, "0x")` → ETH is sent → `Executed` event.
- The subgraph indexes `Executed`; the UI shows the tx hash + Etherscan link.

8.3 Generating & registering a PQC key

- Keys page → **Generate ML-DSA Keypair** → `generateKeypair()` creates the mock FALCON-512 keypair + real ML-KEM keys, encrypted with PBKDF2/AES-GCM, stored in `localStorage`.
- **Register On-Chain** → the page ABI-encodes `setPqcPublicKeyHash(newHash)` (drift — see Section 12), wraps it in a UserOp, ECDSA + PQC signs, and submits through the bundler.
- The contract stores `pqcPubKeyHash` and emits `PqcKeyUpdated`.
- **Test Sign & Verify** → calls `/api/verify-pqc`, which does real ML-DSA-65 verification server-side.

8.4 Uploading an encrypted file (the Vault)

This is where the cryptography is **real**:

- Read the file bytes.
- **ML-KEM-768 encapsulate:** `ml_kem768.encapsulate(mlKemPublicKey)` returns `[sharedSecret, ciphertext]`.
- **AES-GCM encrypt:** use the 32-byte shared secret as an AES-GCM key to encrypt the file

with a random IV.

- (In production you'd upload `{ iv, kemCiphertext, encryptedData }` to IPFS. Here the CID is **mocked** with a random `Qm...` string + a fake delay.)

- **On-chain commit:** `addVaultFile(mockCid)` via a UserOp (ECDSA + PQC signed).
- The subgraph indexes `VaultFileAdded`; the vault list re-renders.

Security model: the file can only be decrypted by someone holding the ML-KEM private key (kept encrypted in the browser keystore). The shared secret itself is only recoverable by decapsulating the ML-KEM ciphertext with that private key — so a "store now, decrypt later" quantum attacker can't read it.

8.5 Social recovery

- Guardians are added via `addGuardian` (through UserOps).
- If you lose your key, a guardian proposes recovery (`initiateRecovery`) with a new owner/PQC key.
- After ≥ 1 day and majority guardian approval, `executeRecovery` rotates the owner + PQC key.

9. The indexing layer (subgraph)

`subgraph/` defines a The Graph subgraph:

- `schema.graphql` entities: `Account`, `Transaction`, `Identity`, `VaultFile`, `Guardian`.
- `subgraph.yaml` declares:
 - a data source for `QuantumSmartWalletFactory` (event `AccountCreated`),
 - a **template** for `QuantumSmartWallet` (events `Executed`, `Deposited`, `IdentityUpdated`, `VaultFileAdded`, `GuardianAdded`).
- `src/mapping.ts`:
 - `handleAccountCreated` saves the `Account` and calls

`QuantumSmartWalletTemplate.create(account)` to start indexing that wallet's events (the standard subgraph "data source template" pattern).

- `handleExecuted` writes **two** `Transaction` records (one "Sent" for the wallet, one "Received" for the target).
- `handleDeposited`, `handleIdentityUpdated`, `handleVaultFileAdded`, `handleGuardianAdded` persist the corresponding entities.

The dApp queries this subgraph (hard-coded URL

`https://api.studio.thegraph.com/query/1757567/quantum/version/latest`) from the Dashboard,

Vault, Security pages, and `/api/activity`.

10. Tooling, testing & CI

- **Hardhat** (`hardhat.config.js`, solc 0.8.24, `evmVersion: cancun`) for JS tests & deployment. `@nomicfoundation/hardhat-foundry` bridges to Foundry.
- **Foundry** (`foundry.toml`, solc 0.8.28) for Solidity fuzz tests. Uses the `forge-std` submodule (currently **empty** — see Section 12).
- **Tests:**
 - `test/QuantumSmartWallet.test.js`: Hardhat tests for owner/entryPoint setup, deposits, `execute`, `validateUserOp` (valid/invalid FALCON mock sig, short sig), and the full social-recovery lifecycle (initiate/approve/timelock/threshold).
 - `testFoundry/QuantumSmartWallet.t.sol`: Foundry tests incl. a fuzz test that any signature < 130 bytes fails.
- **CI** (`.github/workflows/`):
 - `ci.yml`: npm install → ESLint → Hardhat tests → Foundry fuzz tests.
 - `slither.yml`: runs the Slither static analyzer on the contracts.
- **Deploy scripts** (`scripts/`): several variants for Sepolia/local deployment, funding, and auto-updating `constants.js`. Some are outdated (see Section 12).

11. Honest assessment: what is real vs. mocked

As a learner, the single most valuable skill is separating the **claims** from the **code**.

Here is the truth of this repo:

Real

- **ML-KEM-768 (Kyber) keygen + encapsulation** in the Vault — real, from `@noble/post-quantum/ml-kem.js`.
- **AES-256-GCM** file encryption via WebCrypto — real.
- **PBKDF2 + AES-GCM keystore** for protecting private keys in the browser — real.
- **ML-DSA-65 verification** in `/api/verify-pqc` — real (`ml_dsa65.verify`).
- **ECDSA** 2FA via MetaMask — real (standard Ethereum signatures).
- **ERC-4337** UserOp construction, hashing, and `handleOps` submission — real protocol usage against the canonical Sepolia EntryPoint.
- **CREATE2** counterfactual addresses — real.

Mocked / simulated

- **On-chain FALCON-512 verification** — not real. `MockFalconPrecompile` just compares two 32-byte hashes. A real FALCON verifier (or EIP-7619 precompile) is not available on Sepolia.

- **FALCON-512 signing** in `pqcKeyManager.signPayload` — not real. It returns 666 bytes of randomness with an embedded hash; it does not run the FALCON algorithm.
- **QRNG entropy** — fetched from ANU but only XORed into mock key bytes (nice touch, but not the actual FALCON key generation).
- **IPFS upload** in the Vault — mocked (random CID + `setTimeout`). The encryption is real, the "storage" is not wired to a real IPFS node.
- **Dashboard portfolio numbers** (`/api/wallet` returns hard-coded BTC/ETH/QNT/SOL balances) — mock, used when no wallet is connected or the local node is down.
- **ML-DSA precompile path** (`pqcAlgorithmId == 1`) — explicitly `reverts`; not implemented.

Design reality vs. naming

The README/UI mention "ML-DSA-65" in several places, but the browser keypair generated by `generateKeypair` is sized as **FALCON-512** (897-byte pk / 1281-byte sk), and the on-chain path actually used is algorithm **ID 2 = FALCON-512**. This is normal PoC churn — the project evolved from an ML-DSA/oracle design to a FALCON/precompile design, and not every string was updated.

12. Bugs, drift & things to verify

These are exactly the kinds of issues a careful code review (or a learner "reading like a reviewer") should flag:

- `constants.js` **ABI drift**. The `CONTRACT_ABI` references an older contract (`pqcValidator`, `setPqcPublicKeyHash(bytes32)`, `userIdentities`, `encryptedVaultFiles`, `guardians`) that does **not** match the current `QuantumSmartWallet.sol` (`pqcPrecompile`, `setPqcPublicKey(uint8,bytes32)`, `userIdentity`, `vaultFiles`, `guardiansList/isGuardian`, recovery functions). The `FACTORY_ABI` similarly lists a 3-arg `createAccount/getAddress`, but the current factory takes 4 args (`owner`, `uint8 pqcAlgorithmId`, `bytes32 pqcPubKeyHash`, `uint256 salt`).
- `useSmartWallet.js` **calls** `getAddress` **with 3 args** and `/api/bundler/prepare` encodes `createAccount` with 3 args — both inconsistent with the current 4-arg factory. The counterfactual address computed by the UI may therefore not match what the factory would actually deploy.
- `keys/page.js` **calls** `setPqcPublicKeyHash(bytes32)`, but the current contract only has `setPqcPublicKey(uint8, bytes32)`. Key registration would fail against the current contract.
- **Mock FALCON signature hash mismatch**. The client `signPayload` embeds

`keccak256(raw userOpHash)` into the signature's first 32 bytes, but the contract sends `toEthSignedMessageHash(userOpHash)` (the EIP-191-prefixed hash) to the mock precompile.

The two hashes differ, so the mock verification would fail. (The Hardhat test passes only because it manually embeds the prefixed hash.) A real implementation must make the signing convention consistent end-to-end.

- `/api/fund` **imports** `@/utils/activityStore`, **which does not exist** in the repo (`src/utils/` only has `constants.js` and `pqcKeyManager.js`). It's wrapped in `try/catch`, so funding still works — the activity log is silently skipped.
- `/api/wallet` **and** `/api/graphql` **point at** `http://127.0.0.1:8545`, a local Hardhat node that won't exist in production; they silently fall back to mock data.
- `lib/forge-std` **submodule is empty** (not initialized). `forge test` would fail until `git submodule update --init` is run.
- **Stale deploy scripts.** `scripts/deploy.js` and `scripts/deploy.cjs` deploy with an old 2-arg constructor (`Wallet.deploy(pqcPublicKeyMock, deployer.address)`), which no longer matches the 5-arg constructor. `deploySepolia.mjs`/`deployPure.mjs` use 4 args (old signature, missing `pqcAlgorithmId`).
- `hardhat.config.js` **uses solc 0.8.24**; `foundry.toml` **uses 0.8.28** — minor toolchain inconsistency to be aware of when reproducing builds.
- `tmp_abi.json` at the repo root is a UTF-16-encoded ABI dump (looks like binary in a normal editor) — a leftover artifact.
- **Social recovery uses** `(guardians.length / 2) + 1` as a threshold, but an attacker controlling the entrypoint can already do anything; and `hasApprovedRecovery` is never reset between proposals (the code comments acknowledge a `recoveryNonce` would be needed in production).

None of these invalidate the educational value — they are, in fact, excellent study material for learning to audit a real codebase versus trusting its README.

13. How to run it yourself

Prerequisites

- Node.js $\geq$ 18, Git, MetaMask (with a Sepolia account + test ETH),
- an Alchemy Sepolia RPC URL.

Frontend

```
git clone https://github.com/SuhasRam356/Quantum-wallet.git
cd Quantum-wallet
npm install
```

```
# create .env:
#   SEPOLIA_RPC_URL="https://eth-sepolia.g.alchemy.com/v2/YOUR_KEY"
#   SEPOLIA_PRIVATE_KEY="your_test_wallet_private_key"
#   VALIDATOR_PRIVATE_KEY="optional"
npm run dev
# open http://localhost:3000
```

Contracts (Sepolia)

```
node scripts/deploySepolia.mjs # deploys a QuantumSmartWallet
# (or deploy the Factory first, then create wallets through it)
```

Subgraph

```
cd subgraph
npm install
npx graph auth --studio YOUR_DEPLOY_KEY
npm run codegen && npm run build
npx graph deploy --studio quantum-wallet --version-label 0.0.1
```

Tests

```
npx hardhat test # JS tests
git submodule update --init && forge test # Foundry fuzz tests (need forge-std)
```

14. Glossary

Term	Meaning
EOA	Externally Owned Account (a normal private-key wallet like MetaMask).
Smart Contract Wallet	A wallet whose logic is a contract; can enforce custom rules.
ECC / ECDSA	Elliptic Curve Cryptography / Digital Signature Algorithm — today's standard.
Shor's algorithm	Quantum algorithm that breaks RSA/ECC.
PQC	Post-Quantum Cryptography — algorithms safe against quantum computers.
ML-KEM / Kyber	NIST-standard lattice KEM (encryption/key exchange).
ML-DSA / Dilithium	NIST-standard lattice signature scheme.
FALCON / FN-DSA	NIST-standard lattice signature scheme with small signatures.
ERC-4337	Account Abstraction standard using UserOps, EntryPoint, Bundlers.
UserOp	A "pseudo-transaction" processed by the EntryPoint.
EntryPoint	Singleton contract that validates/executes UserOps.
Bundler	Off-chain service that submits UserOps to the EntryPoint.
Paymaster	Contract that sponsors gas for UserOps.
CREATE2	Opcode enabling deterministic (predictable) contract addresses.
Counterfactual address	An address computed before the contract is deployed.
Subgraph	The Graph indexer that turns contract events into a queryable DB.
IPFS / CID	Decentralized storage / content-addressed identifier.
Precompile	A native (protocol-level) contract, e.g. for fast crypto.
Crypto-agility	Ability to swap cryptographic algorithms without redeploying.

Disclaimer (from the README): this is an educational proof-of-concept. The cryptography research is real, but the contracts are built for test environments. Do not store real mainnet funds in this wallet yet.